SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I **VENNA MEENAKSHI REDDY – 192471048** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **VENNA MEENAKSHI REDDY**

RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

QUESTION 1

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

ANSWER:

1. Identify appropriate HTML5 semantic elements missing in the structure.

The given HTML form is functional but does not use HTML5 semantic elements, making it less structured and less accessible.

Missing semantic elements:

- <header> – Displays the portal title or college logo.
- <main> – Contains the main content of the webpage.
- <section> – Groups the symposium registration content.
- <article> – Represents the registration form as an independent component.
- <footer> – Displays copyright or contact information.
- <label> – Provides accessible labels for input fields.
- <fieldset> and <legend> – Groups related form controls with a meaningful title.

These semantic elements improve document structure, accessibility, SEO, and maintainability.

2. Modify the structure to make it standards-compliant.

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Online Symposium Registration</title>
</head>

<body>

<header>
   <h1>College Online Symposium Registration</h1>
</header>

<main>

<section>

<article>

<form action="/register" method="POST">

<fieldset>

<legend>Student Registration Form</legend>

<label for="name">Full Name</label>
<input type="text"
      id="name"
      name="name"
      required>

<br><br>

<label for="email">Email Address</label>
<input type="email"
      id="email"
      name="email"
      required>

<br><br>

<button type="submit">
Submit
</button>

</fieldset>

</form>

</article>
```

```
</section>

</main>

<footer>
<p>&copy; 2026 College Symposium Portal</p>
</footer>

</body>
</html>
```

Standards followed

- HTML5 <!DOCTYPE html>
- Language declared using lang="en"
- UTF-8 character encoding
- Responsive viewport meta tag
- Semantic HTML elements
- Proper labels associated with inputs
- Required field validation
- Explicit type="submit" button

3. Interpret the HTTP request generated during form submission.

When the student clicks Submit, the browser creates an HTTP POST request because the form uses method="POST".

Example HTTP Request

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

Content-Length: 33

name=Rahul&email=rahul@example.com

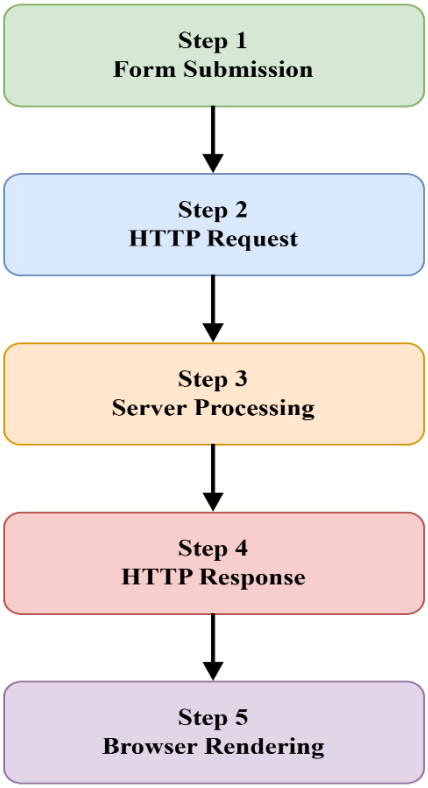
Interpretation

- **POST** → Sends data securely in the request body.
- **/register** → Server endpoint handling registration.
- **HTTP/1.1** → Protocol version.
- **Host** → Identifies the destination server.
- **Content-Type** → Indicates form data format.
- **Content-Length** → Specifies the size of transmitted data.
- **Request Body** → Contains the student's registration details (name and email).

Since POST stores data in the request body, sensitive information is not exposed in the URL, making it more suitable than GET for registration forms.

4. Analyze the HTTP response cycle from server to browser.

The complete HTTP response cycle is:



Step 1 – Form Submission

The student enters the name and email and clicks Submit.

↓

Step 2 – HTTP Request

The browser sends a POST request containing the form data to /register.

↓

Step 3 – Server Processing

The server:

- Receives the request.
- Validates the input.
- Stores the registration details in the database.
- Generates an appropriate response.

↓

Step 4 – HTTP Response

Example:

HTTP/1.1 200 OK

Content-Type: text/html

<h2>Registration Successful</h2>

Possible response status codes include:

- **200 OK** – Registration completed successfully.
- **201 Created** – New registration record created.
- **400 Bad Request** – Invalid input.
- **404 Not Found** – Registration endpoint not found.
- **500 Internal Server Error** – Server-side failure.

Step 5 – Browser Rendering

The browser:

- Receives the response.
- Parses the HTML.
- Builds the DOM.
- Renders the confirmation page or error message for the student.

5. Suggest improvements for usability and accessibility.

Usability Improvements

- Add meaningful placeholders (e.g., "Enter your full name").
- Display clear validation and error messages.
- Use responsive design for mobile devices.
- Provide a confirmation message after successful registration.
- Disable multiple submissions while processing.

Accessibility Improvements

- Associate every input with a <label>.
- Use <fieldset> and <legend> to group related controls.
- Mark mandatory fields using the required attribute.
- Ensure keyboard accessibility (Tab navigation).
- Maintain sufficient color contrast.
- Use semantic HTML5 elements for screen readers.
- Add ARIA attributes where necessary for dynamic content.

Conclusion

The improved HTML structure follows HTML5 standards by using semantic elements, proper form controls, and accessibility features. The form submits data using an HTTP POST request, the server processes the request and returns an appropriate HTTP response, and the browser renders the resulting page. These enhancements improve standards compliance, usability, accessibility, maintainability, and overall user experience, resulting in a secure, efficient, and standards-compliant web application.

QUESTION 2

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

ANSWER:

1. Identify improper nesting and missing closing tags.

The given HTML contains several syntax and structural errors that may lead to inconsistent rendering across browsers.

Errors Identified

1. Missing <!DOCTYPE html> declaration.
2. Missing lang attribute in the <html> element.
3. <h1> opening tag is not closed (</h1> missing).
4. opening tag is not closed (missing).
5. contains improperly nested elements because the <h1> is not closed before the begins.
6. No semantic HTML5 elements such as <header> and <main>.

Incorrect Structure

```
<div>
<h1>Welcome
<p>About us
</div>
```

Correct Nesting

```
<div>
  <h1>Welcome</h1>
  <p>About us</p>
</div>
```

2. Analyze how different browsers may interpret this structure.

Browsers use HTML parsing algorithms to automatically correct invalid HTML. However, each browser's rendering engine may repair errors differently, leading to inconsistent layouts.

Browser Interpretation

Google Chrome (Blink Engine)

- Automatically closes the unclosed <h1> before starting the <p> element.
- Displays the page correctly in most cases.
- Minor layout issues may still occur.

Mozilla Firefox (Gecko Engine)

- Attempts to repair the invalid HTML.
- Usually produces a layout similar to Chrome.
- DOM structure may differ internally.

Microsoft Edge (Blink Engine)

- Similar behavior to Chrome because both use the Blink rendering engine.
- Automatically inserts missing closing tags.

Safari (WebKit Engine)

- Repairs invalid HTML during parsing.
- May apply different spacing or styling because of WebKit's rendering behavior.

Effect on Cross-Browser Compatibility

Because browsers repair malformed HTML differently:

- Layout may appear inconsistent.
- CSS rules may apply differently.
- JavaScript DOM selection may behave unexpectedly.
- Accessibility tools may interpret the document incorrectly.

Using valid W3C-compliant HTML ensures consistent rendering across all browsers.

3. Identify missing semantic elements (e.g., header, section).

The webpage lacks HTML5 semantic elements that improve structure, accessibility, and SEO.

Missing Semantic Elements

- <header> – Contains the company title or logo.
- <main> – Represents the primary page content.
- <section> – Groups related information.
- <article> – Represents an independent content block (optional).
- <footer> – Displays copyright or contact details.

Example Semantic Structure

<header>
↓
<main>
↓
<section>
↓
<h1>
<p>
↓
<footer>

Using semantic elements improves:

- Accessibility for screen readers.
- Search engine optimization (SEO).
- Code readability.
- Website maintainability.

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Company</title>
</head>

<body>

<header>
   <h1>Welcome</h1>
</header>

<main>

<section>

<p>About us</p>

</section>

</main>

<footer>
   <p>&copy; 2026 Company. All Rights Reserved.</p>
</footer>

</body>

</html>
```

Why this is W3C-compliant

- Uses <!DOCTYPE html>
- Includes lang="en"
- Properly closes all tags
- Uses semantic HTML5 elements
- Includes UTF-8 encoding
- Includes responsive viewport meta tag
- Maintains proper document hierarchy and nesting

5. Suggest techniques to ensure cross-browser compatibility.

Recommended Techniques

1. **Write valid HTML5**
    - Follow W3C standards.
    - Close all tags correctly.
    - Use proper nesting.
2. **Use semantic HTML elements**
    - Improves browser interpretation and accessibility.
3. **Validate HTML and CSS**
    - Check code using W3C HTML and CSS validators.
    - Fix syntax errors before deployment.
4. **Use CSS Reset or Normalize.css**
    - Reduces browser-specific default styling differences.
5. **Test on multiple browsers**
    - Verify the website on Chrome, Firefox, Edge, Safari, and mobile browsers.
6. **Use responsive design**
    - Apply CSS media queries.
    - Include the viewport meta tag.
7. **Use standard CSS properties**
    - Avoid deprecated or browser-specific features unless necessary.
    - Provide fallbacks where appropriate.
8. **Test JavaScript compatibility**
    - Use modern JavaScript features with polyfills or transpilers when required.
9. **Use external CSS and JavaScript files**
    - Improves maintainability and consistent loading across browsers.
10. **Perform regular compatibility testing**
    - Test after updates to ensure consistent rendering and functionality.

Conclusion

The corrected HTML follows W3C standards by using proper tag nesting, closing all elements, and incorporating HTML5 semantic tags. This ensures consistent rendering across different browsers, improves accessibility and SEO, and enhances code readability and maintainability. Applying validation, responsive design, and cross-browser testing techniques results in a reliable, professional, and standards-compliant website that delivers a consistent user experience across all modern browsers.

QUESTION 3

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

ANSWER:

1. Identify issues in the form configuration.

The given HTML form contains several configuration issues that prevent proper form submission and compromise security.

Issues Identified

1. The form uses the GET method to transmit login credentials.
    - Passwords should never be sent using GET because they become visible in the URL.
2. The button type is button instead of submit.
    - A button with type="button" does not submit the form unless JavaScript is used.
3. Input fields do not have the required attribute.
    - Empty values can be submitted.
4. The password field lacks security-related attributes such as autocomplete="current-password" (recommended).
5. The form does not include semantic labels (<label>), reducing accessibility.

Given Configuration

<form action="/submit" method="GET">
<input type="text" name="username">
<input type="password" name="password">
<button type="button">Login</button>
</form>

2. Analyze why the HTTP request is not triggered.

An HTTP request is **not triggered** because the button is defined as:

<button type="button">Login</button>

Explanation

- type="button" creates a normal button.
- It performs no default action.
- Clicking the button does not submit the form.
- Therefore, the browser never sends an HTTP request to /submit.

An HTTP request is generated only when:

- The button type is submit, or
- JavaScript explicitly calls form.submit().

Since neither occurs, the server never receives the form data.

3. Interpret the role of GET method in this scenario.

The **GET** method appends form data to the URL as query parameters.

Example HTTP GET Request

GET /submit?username=Rahul&password=abc123 HTTP/1.1

Host: www.example.com

Interpretation

- Form data is attached to the URL.
- Username and password become visible in:
    - Browser address bar
    - Browser history
    - Server logs
    - Bookmarks
    - Proxy caches

Why GET is unsuitable

- Login credentials are sensitive.
- URLs can be exposed or stored.
- GET should be used only for retrieving data, not for authentication or confidential information.

For login forms, POST is the appropriate HTTP method because it sends data in the request body instead of the URL.

4. Propose a corrected implementation.

```
<!DOCTYPE html>
<html lang="en">

<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Login Form</title>
</head>

<body>

<form action="/submit" method="POST">

<label for="username">Username</label>
<input type="text"
      id="username"
      name="username"
      required>

<br><br>

<label for="password">Password</label>
<input type="password"
      id="password"
      name="password"
      autocomplete="current-password"
      required>

<br><br>

<button type="submit">
Login
</button>

</form>

</body>

</html>
```

Corrections Made

- Changed method="GET" to method="POST".
- Changed type="button" to type="submit".
- Added <label> elements.
- Added required validation.
- Added autocomplete="current-password".
- Included HTML5 document structure.

5. Suggest improvements for secure data transmission.

Security Improvements

1. **Use POST instead of GET**
    - Sends form data in the request body instead of the URL.
2. **Use HTTPS**
    - Encrypts data exchanged between the browser and server using SSL/TLS.
3. **Validate input**
    - Perform both client-side and server-side validation.
4. **Use strong authentication practices**
    - Store passwords securely using hashing algorithms (e.g., bcrypt or Argon2) on the server.
5. **Protect against CSRF attacks**
    - Include CSRF tokens in forms.
6. **Use secure session management**
    - Generate secure session IDs after successful login.
7. **Limit login attempts**
    - Prevent brute-force attacks by implementing account lockout or rate limiting.
8. **Avoid exposing sensitive information**
    - Never include passwords in URLs or log files.
9. **Use appropriate security headers**
    - Configure headers such as Content-Security-Policy and Strict-Transport-Security.
10. **Provide user-friendly error messages**
    - Display generic login failure messages without revealing whether the username or password is incorrect.

Conclusion

The original form fails to submit because the button is defined as type="button", which does not trigger form submission, and the use of the GET method exposes sensitive information in the URL. Replacing GET with POST, using a submit button, adding proper labels and validation, and implementing secure transmission through HTTPS and server-side security measures ensures reliable, secure, accessible, and standards-compliant form submission.